INTERFACE.AI TAKE-HOME · ASSIGNMENT A

Computer-Use Automation System

An AI learns a task once in a legacy bank screen. It is then replayed exactly, with no AI, every time an agent asks for it, and a person steps in whenever judgment is needed.

End-to-end overview: what the assignment asks for, what was built and why, how it works, the results, and what was left out. 11 October 2026.

29 / 29	**64**	**10 / 10**	**0**
scenario cases produce exactly the expected result	automated tests, all passing	replays per approved version in stability runs	LLM calls in the production replay path

1 The problem in plain words
2 The end-to-end flow
3 Requirements checklist
4 Architecture and key choices
5 The capability artifact
6 Deterministic replay and errors
7 Human escalation and handoff
8 Safety and data handling
9 Other surfaces and many tenants
10 Results and evidence
11 Every case, with its result
12 Screens
13 What was cut, and honest gaps
14 How to run it

The brief's through-line: *"The model discovers. The artifact becomes a reusable capability. Deterministic replay is how the AI agent invokes it in production."* Everything below is organised around that sentence.

1 The problem in plain words

Banks and credit unions run many old internal applications: teller consoles, servicing tools, admin screens. Most have **no API**, so the only way to get work done in them is to click through the screens like a staff member would.

interface.ai builds AI agents for those institutions. An agent might decide "look up this member's balance" or "open a sub-account for this member", but it needs **hands** to actually do that inside the old software. This project is those hands.

Why not let the AI click every time?

It is slow, costly, and can behave differently from one run to the next. In banking, "usually right" is not acceptable, and every action must be explainable afterwards.

Why not hand-write scripts?

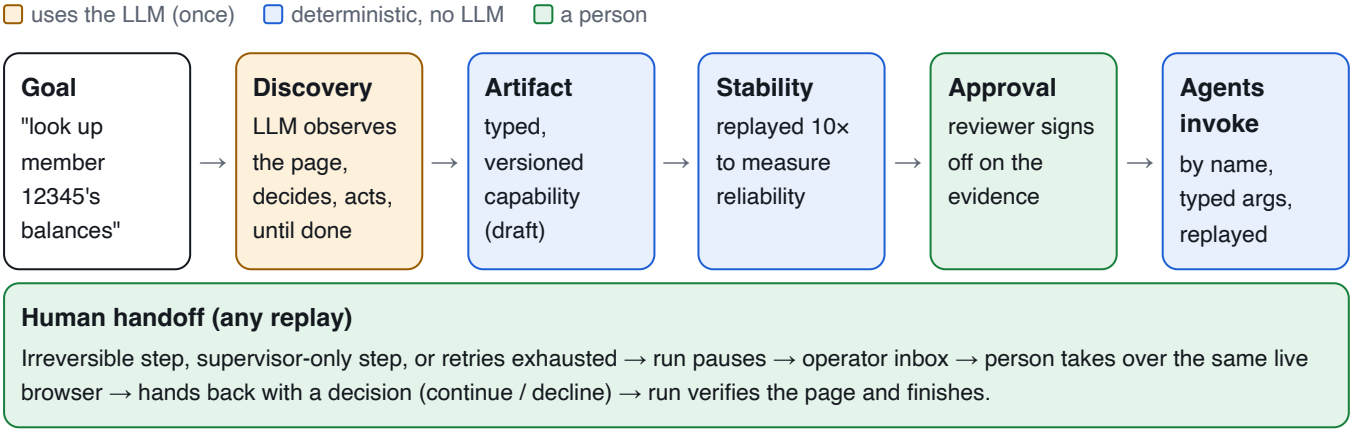
There are hundreds of institutions running about 20 apps each, thousands of app instances. Hand-scripting each flow does not scale, and scripts break silently on the unhappy paths.

The approach

1. **Learn once.** An LLM is given a goal in plain English and drives a real browser until the task is done.
2. **Save it as a contract.** What it did is saved as a typed, versioned **capability**: what inputs it needs, what it returns, how each control on screen is found, what can go wrong, and how success is checked.
3. **Replay exactly.** From then on the capability runs with **no AI**: same inputs, same steps, same checks, every time. Fast, cheap and auditable.
4. **Bring in a person when needed.** If a step is irreversible, needs a supervisor, or the app misbehaves in a way automation cannot fix, the run pauses and a person takes over the *same live session*, then hands control back.
5. **Stay safe throughout.** An allowlist limits where it can go and what it can do; sensitive data is masked in everything written to disk.

> **The key distinction the brief stresses:** "no such member" is a valid *answer*, not a crash. The system keeps three kinds of result apart: business outcomes (tell the caller), recoverable conditions (handle and continue), and hard failures (stop with a clear, debuggable error).

2 The end-to-end flow



Every run, of any kind, writes evidence: a structured log, a state-machine record, screenshots, and one line in a ledger that reliability scores are computed from.

One real example, start to finish

1. **Discovery** (`evidence/discovery-3EvQLgjD`): goal "look up member 12345 and read their savings and checking balance". Model `cohere/north-mini-code:free` took 6 tool-calling turns: fill the member ID, click Look Up Member, read Savings, read Checking, finish.
2. **Artifact**: saved as `lookup_member_balance` v1, with `memberId` as a typed, sensitive input and two typed outputs. The recorded ID became a `{{memberId}}` parameter.
3. **Revisions**: v2 added a compliance-notice check; v3 made the success check host-independent. Each new version had to earn approval again from zero.
4. **Stability**: v3 replayed 10/10 at about 730 ms each, giving a 72% confidence lower bound, which passes the 70% policy. Approved.
5. **Use**: an agent calls `lookup_member_balance({memberId:"23456"})` and gets `{savings_balance, checking_balance}` back, or a named outcome such as `MEMBER_NOT_FOUND` .
6. **Another tenant**: the same v3 file ran unchanged against a second copy of the app on a different host, where a button had been renamed. A fallback locator found it and the drift was recorded (`evidence/replay-x9ucrkE0`).

3 Requirements checklist

Each must-have from section 3 of the brief, what was built for it, and where to see it.

REQUIREMENT	WHAT WAS BUILT	PROOF	STATUS
3.1 Goal-driven agent loop goal + target in, observe → decide → act, stop conditions	CLI `discover` takes a goal and entry point. The loop shows the model an accessibility-style snapshot (roles, names, label text) and offers tools: click, fill, select_option, extract_by_label, escalate, finish. It stops at `finish` , at 20 turns, on timeouts, or on escalation. Chosen because it works on markup with no clean DOM or test IDs.	`src/llm/` , `src/perception/` , `evidence/discovery-3EvQLgjD`	**Done**
3.2 Structured artifact steps, how controls are found, typed inputs/outputs, checkpoint, versioned, reviewable	A JSON capability contract validated by a schema: typed inputs (with pattern and sensitive flag), typed outputs, ranked locator plans with a rationale, named outcome checks, a success checkpoint, an irreversible-step type, risk level, and provenance (model, run, reviewer, approval evidence). Contract checks refuse to save an artifact that ignores an input.	`src/artifact/schema.ts` , `artifacts/*.json`	**Done**
3.3 Deterministic replay no LLM, stable targeting, checkpoint, outputs, error taxonomy, debuggable result	The replay engine never imports the LLM. Locators resolve in fixed order and need exactly one match. Outcomes come from the real HTTP status. Results are `success` , `business_outcome` , `failure` (step, expected, observed, code) or `rehearsal_complete` .	`src/replay/` , 29 scenarios, replay evidence folders	**Done**

REQUIREMENT	WHAT WAS BUILT	PROOF	STATUS
3.4 Safety and guardrails allowlist, risky vs. safe, never persist secrets/PII	A configurable allowlist of origins and action types, checked on every navigation and click. Irreversible steps always pause for a person unless separately cleared. Redaction masks IDs, account and card numbers, SSNs and sensitive input values everywhere; passwords and PINs are never stored.	`config/allowlist.json`, `src/guardrails/`, cases 21–22	**Done**
3.5 Evidence structured log + a richer failure signal	Each run writes `log.jsonl`, `run.json` (state transitions, step timings, drift, handoffs), the result, and screenshots at the final state, at every pause and on crash.	`evidence/`	**Done**
3.6 Human escalation and handoff detect and route, same live session, hand back, record what the human did	Four trigger kinds route to one mechanism. The operator gets an inbox and a takeover console that acts on the *same* browser page, with every human action recorded. Hand-back carries a decision; the engine checks the page instead of trusting it. Unanswered pauses time out.	`src/escalation/`, cases 9–17	**Done**
3.7 Heterogeneity and scale design, not necessarily build	Design: perception produces the same shape from DOM or an OS accessibility tree; a new surface adds a locator kind, not a new format. Multi-tenant: base artifact plus per-tenant overrides. Partly built: run-time origin, tenant skins, drift detection.	REPORT.md §4, cases 2 and 19, `replay-x9ucrkE0`	**Designed**

Deliverables (brief section 6)

DELIVERABLE	WHERE	STATUS
README with setup, keys, demo path	`README.md`: install, `.env` key, discover → replay → stability → approve → server commands	**Done**
REPORT.md, seven exact headings	`REPORT.md`: Architecture, Artifact schema, Determinism & error handling, Heterogeneity & multi-tenant, Escalation & handoff, Safety, Cuts	**Done**
Evidence: artifact, discovery log, replay log, one error replay	`evidence/`: genuine discovery run, many replays including not-found, permission denied, validation error, retries; a guide in `evidence/README.md`	**Done**
Public GitHub repository	Not pushed yet; waiting on the author's go-ahead	**Pending**

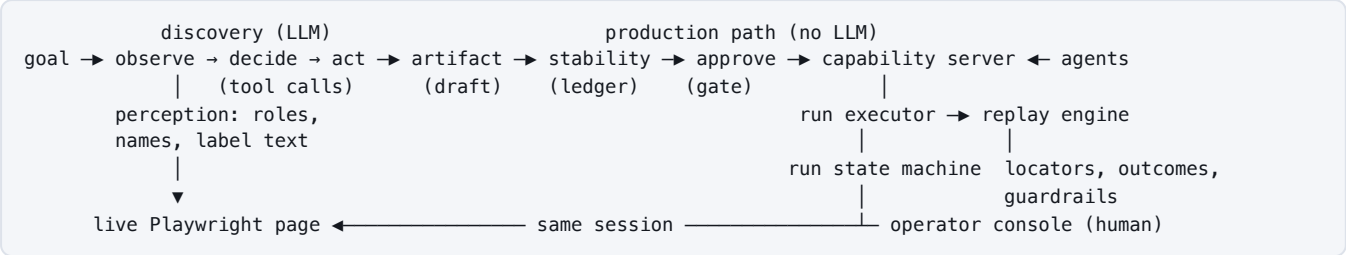
Stretch goals (brief allows one or two)

STRETCH GOAL	WHAT WAS BUILT
Agent-facing capability interface	An HTTP server lists approved capabilities as OpenAI or Anthropic tool definitions generated from the artifacts, and runs them by name with typed arguments, idempotency keys and long-polling. Shown in a scripted API walkthrough.

STRETCH GOAL	WHAT WAS BUILT
Confidence and approval	Every run feeds a ledger. Reliability is the Wilson 95% lower bound, not the raw rate. Agents can only call approved versions; approval needs 8+ runs at 70%+ confidence. Risky flows earn it in rehearsal mode without doing anything irreversible.

Multi-run stability is used as the evidence feeding approval, not as a third separate goal. Cross-tenant replay is partly shown (run-time origin, fallback locators on a re-labelled tenant) but per-tenant overrides were not built.

4 Architecture and key choices



Components

PART	JOB	CODE
Stand-in bank app	A deliberately legacy teller console: table layouts, no test IDs. It can produce each runtime error on demand.	`src/target-app/`
Perception	Turns the live page into numbered elements with role, name, label text and candidate locators.	`src/perception/`
Discovery loop	Model in the loop: observe → decide → act, then records the artifact.	`src/llm/`
Artifact store and checks	Schema, versioned files, contract validation.	`src/artifact/`
Replay engine	Runs an artifact with no model: locators, outcomes, recovery, checkpoints.	`src/replay/`
Run executor and state machine	One path for every run (CLI, stability, server): fresh browser, session, evidence, ledger.	`src/runs/`
Escalation	Pause, hand control to a person on the same page, record actions, resume.	`src/escalation/`
Guardrails	Allowlist, risk policy, redaction.	`src/guardrails/`
Capability server	Agent API, operator inbox, control center, roles and sign-in.	`src/server/`

Choices and why

DECISION	CHOICE	WHY
Language	TypeScript on Node	Strong types for the artifact contract; Playwright's first-class home.

DECISION	CHOICE	WHY
Surface control	Playwright, driven through an accessibility-style snapshot, not pixel coordinates	Works on legacy markup with no test IDs, replays deterministically, and has the same shape as an OS accessibility tree on desktop.
LLM	OpenRouter, OpenAI-style tool calling, free model `cohere/north-mini-code:free`	Chosen by testing current free models on real observations from the app. Any tool-calling model can be swapped in by setting one variable.
Target app	A local mock legacy console	Can trigger every error the brief names (not found, denied, timeout, session expiry, dialogs) on demand, and uses no real bank or real PII.
Architecture	One process, no queues	The brief does not reward scaling infrastructure. Boundaries are kept sharp in code instead.
Validation	zod schemas	One source of truth for runtime checks and types.

> Discovery and replay share only perception and locator code. The replay engine never calls the LLM, so "no model in the production loop" holds by construction, not by convention.

5 The capability artifact

The artifact is a **contract an agent can call**, not a step recording. A reviewer and a calling agent can both see what it does, what it needs and what it returns without reading the steps.

A real artifact (abridged): `artifacts/lookup_member_balance.v3.json`

```
{
  "name": "lookup_member_balance", "version": 3, "riskLevel": "safe",
  "target": { "appName": "Meridian Teller Console", "entryPathTemplate": "/members/search" },
  "inputParams": [{ "name": "memberId", "type": "string", "pattern": "^\\d+$", "sensitive": true }],
  "outputs": [
    { "name": "savings_balance",  "type": "string", "description": "Current savings balance" },
    { "name": "checking_balance", "type": "string", "description": "Current checking balance" } ],
  "successCheckpoint": { "kind": "urlMatches", "pattern": "/members/[^/]+$" },
  "steps": [
    { "type": "fill", "description": "Enter member ID {{memberId}} ..." },
    { "type": "click", "target": {
        "strategies": [
          { "kind": "role", "role": "button", "name": "Look Up Member" },
          { "kind": "text", "text": "Look Up Member" },
          { "kind": "css",  "selector": "body form > table > tbody > tr:nth-of-type(2) > td > input" } ],
        "rationale": "role+accessible name is stable across re-skins; css path kept as fallback" } },
    { "type": "checkOutcomes", "outcomes": [
        { "code": "MEMBER_NOT_FOUND",  "category": "business_outcome", "detect": { "httpStatus": 404 } },
        { "code": "PERMISSION_DENIED", "category": "business_outcome", "detect": { "httpStatus": 403 } },
        { "code": "CORE_TIMEOUT",      "category": "recoverable",      "detect": { "httpStatus": 504 },
          "recovery": { "action": "reload", "maxAttempts": 2, "waitMs": 1000 } },
        { "code": "COMPLIANCE_NOTICE", "category": "recoverable", "recovery": { "action": "dismiss" } } ] },
    { "type": "extract", "label": "Savings" },
    { "type": "extract", "label": "Checking" } ],
  "provenance": {
    "discoveryRunId": "3EvQLgjD", "model": "cohere/north-mini-code:free", "revisedFrom": 2,
    "approval": { "evidence": { "runs": 10, "confidence": 0.722, "mode": "live" } } }
}
```

Shortened for print: `detect` and `extract` fields are condensed. The file itself has the full form.

Why it is shaped this way

- **Typed inputs and outputs** with patterns and a `sensitive` flag, so a caller knows what to pass and the system knows what to mask.
- **A ranked locator plan per control**, in this order: role + accessible name, form-field name, visible text, table cell found by its row label, then a CSS path last. It must match exactly one element. The winning strategy is logged, so drift is visible. "Table cell next to this label" is there because that is the dominant legacy pattern.
- **The error taxonomy lives in the schema**: each expected state is a named outcome with a category, a detection rule and, if recoverable, a recovery policy. Errors are designed in, not handled as exceptions after the fact.
- **Irreversible actions are their own step type** (`confirmIrreversible`), so policy can single them out exactly.
- **Provenance and approval** record which run and model produced it, who reviewed it, and the evidence it was approved on.
- **Login is never a step**: credentials must not enter a versioned file. The system signs in before each run.
- **Recorded values become parameters**: the seed member ID becomes `{{memberId}}`, so no PII is stored in the artifact.

Versioning and the approval gate

Approved versions are never edited in place. `npm run revise` creates a new draft from the recorded steps. Agents are always served the newest *approved* version, so a draft never replaces a working capability until it has earned approval on its own evidence.

CAPABILITY	VERSIONS	AGENTS USE
`lookup_member_balance`	v1 from discovery → v2 added the compliance-notice check → v3 made the success check host-independent	v3
`open_member_subaccount`	v1 from discovery → v2 added supervisor-override and compliance checks	v2

> **A real bug these checks caught:** the model typed the deposit `"60.00"` as the number `60`. The recorder didn't match it to the parameter, so the artifact would have deposited $60 on every run whatever the caller asked for. The recorder now compares numbers by value, and the contract check refuses to save any artifact where an input drives no step.

6 Deterministic replay and errors

How replay stays deterministic

1. It always starts from the artifact's declared entry point, at the origin configured for that run.
2. Locators resolve in a fixed order and must match **exactly one** element. Two matches count as none: on a financial screen it never guesses.
3. Outcomes are detected from the **real HTTP status** of the last navigation, not by scraping page text.
4. Phases are explicit: validate inputs and policy → execute steps → verify the success checkpoint → return outputs.
5. No LLM is called anywhere on this path.

The result contract

RESULT	MEANING	EXAMPLES
success	Done, with typed outputs	`{savings_balance:"$4,823.10", ...}`
business_outcome	A valid answer the caller must know about. Not an error.	`MEMBER_NOT_FOUND`, `PERMISSION_DENIED`, `VALIDATION_ERROR`, `DECLINED_BY_OPERATOR`
failure	Stopped. Says which step, what was expected, what was observed, and a code.	`LOCATOR_NOT_FOUND`, `SESSION_EXPIRED`, `RECOVERY_EXHAUSTED`, `INVALID_PARAMS`, `ORIGIN_NOT_ALLOWED`, `HUMAN_TASK_INCOMPLETE`, `OPERATOR_TIMEOUT`
rehearsal_complete	Ran up to the irreversible step, proved its control exists, and stopped before doing it	Used to earn approval for risky flows safely

"Waiting for a person" is deliberately *not* a result. It is a live run state (`AWAITING_HUMAN`) that callers poll.

Recoverable conditions

RECOVERY	USED FOR	EXAMPLE
wait	A slow load	Give the page more time
reload	A transient error	Core system returns 504 once; reload, then success (case 7)
dismiss	A *known* interstitial only	Acknowledge a compliance notice (case 8). Unknown screens are never dismissed blindly.
human	Something only a person may do	Supervisor override PIN for deposits of $10,000+ (case 12)

If retries run out, the run **escalates to a person** before failing. Only if the person cannot fix it does it end as `RECOVERY_EXHAUSTED`.

Deliberate safety choices

- **Whole runs are never retried automatically.** Re-running a banking flow from the top could do it twice. Retries happen per step only.
- **Agent retries are safe through idempotency keys**: the same key returns the original run, and the same key with different input gets `409`.
- **Drift is detected, not "self-healed".** The brief says these UIs are stable, so any fallback-locator win is logged and counted per version. A change that no fallback covers stops with a clear error instead of guessing (case 19).

7 Human escalation and handoff

When a person is brought in

KIND	TRIGGER	COUNTS AGAINST RELIABILITY?
policy	An irreversible step (e.g. "Confirm open account") on a version not cleared for unattended execution	No: the system working as designed
task	A step **only a person may do**: a supervisor's override PIN. Automation never holds or types it.	No: by design
fault	A recoverable condition that outlived its retries	Yes
agent	The discovery model calls `escalate` because it is stuck	Yes

How control moves

`INITIALIZED` → `RUNNING` ⇄ `AWAITING_HUMAN` → `COMPLETED` | `BUSINESS_OUTCOME` | `FAILED`

1. **Pause.** The run captures a screenshot and the element map, records the capability, step and reason, moves to `AWAITING_HUMAN` and blocks. The record names who holds control.
2. **Route.** The request lands in the operator **inbox** in the control center, with the reason and how long it has waited.
3. **Take over.** "Take over" opens a console for that run, showing the **same live browser page**, refreshed on each poll, and its controls. Every click and fill runs on that exact page and is recorded. Password fields are recorded only as `[REDACTED]`.
4. **Hand back with a decision.** *Resume* or *Decline*. The engine then **checks the page** rather than trusting the operator:
   - If the confirm control is still on screen, the action did not happen, so the result is `DECLINED_BY_OPERATOR`. This rule came from a real session where an operator pressed Resume without confirming.
   - If the supervisor screen is still there, the task was not done, so the result is `HUMAN_TASK_INCOMPLETE`.
   - If nobody answers within 15 minutes, the result is `OPERATOR_TIMEOUT`, so no browser is held forever.
5. **Agent side.** The calling agent receives `202 AWAITING_HUMAN` and long-polls until the person decides. It can never approve its own action (case 28).

One run can need several handoffs. The $15,000 deposit pauses first for the supervisor PIN (`task`), then again for the irreversible confirmation (`policy`).

Simplified on purpose, as the brief's scope note allows: the "live view" is a polled screenshot, not a real-time co-browsing stream. The control-transfer mechanism itself (one lock, one page, decisions, verification, recorded actions) is real.

8 Safety and data handling

Allowlist

`config/allowlist.json` lists allowed origins and action types. It is checked on every navigation and after every click. Leaving it is a hard failure (case 21: a tampered artifact tries to navigate elsewhere and is stopped).

Risky actions

Irreversible steps **require human confirmation** on every run, even after approval. Running them unattended is a separate grant that needs 8 live, human-confirmed runs. Chosen over "block" because the work still has to get done, and over "flag" because a flag after the fact cannot undo an opened account.

Approval gate

Agents can only call approved versions. Approval needs 8+ runs with a Wilson 95% lower bound of 70%+. A raw rate misleads: 3/3 looks "100%" on almost no evidence, while 10/10 only proves "at least 72%".

Separation of duties

Two roles. An **agent** key can list and call approved capabilities and poll its own runs. An **operator** signs in and handles the inbox and takeovers. Approval is a CLI action outside the API. There is no self-registration: in a bank, staff identity comes from the institution's identity provider.

Redaction

- Applied to everything written to disk or the console. Callers still receive real outputs.
- Credential-shaped keys are removed entirely. Identifier keys are masked to the last 4 digits (member 23456 is logged as `*3456`).
- SSN-, card- and account-number-shaped text is masked, even inside an LLM's free-text reasoning.
- Values of inputs marked `sensitive` are masked wherever they appear, including when the model sends them as numbers.
- Password-field values are never put in an LLM prompt and never recorded. The supervisor PIN appears in no evidence file.

Input and load limits

Unknown or malformed parameters are rejected with `400` before a browser starts. Malformed JSON gets `400`. At most 4 browser runs at once; beyond that, invoke returns `429` with `Retry-After`.

Known limits

- Redaction is pattern- and value-based, so a person's *name* in free text is not caught.
- Screenshots inherently contain PII. In production they belong in encrypted, access-controlled, short-retention storage.
- Roles are shared keys, so the audit trail says "an operator", not which one. Production would put SSO in front with the same role split.
- Idempotency keys and live run records are in memory and do not survive a restart. The ledger and evidence are on disk.

9 Other surfaces and many tenants

Legacy web and desktop apps

The seam sits between **perception** (which turns whatever is on screen into `{role, name, candidate locators}`) and the **artifact's locator plan**.

- **Legacy web** needs nothing new: the stand-in app *is* one, with table layouts and no test IDs.
- **Desktop** swaps the DOM query for the OS accessibility tree (UI Automation on Windows, AXUIElement on macOS), which gives the same shape. The artifact format, replay engine, outcome taxonomy, state machine and handoff stay the same. The locator plan only gains a new strategy kind, such as `automationId`.

A new surface adds a locator kind, not a new artifact format.

Hundreds of tenants on the same vendor product

PIECE	STATE
Artifacts hold paths, not hosts; the origin is chosen at run time	Built The same approved v3 ran unchanged against a second instance (`replay-x9ucrkE0`).
Tenant variations of the same app	Built The stand-in app has three skins. Tenant B renames a button, and a fallback locator absorbs it with the drift recorded (case 2). Tenant C renames a data row, and replay stops with a clear error instead of guessing which number is the balance (case 19).
Base artifact + per-tenant overrides	Designed Record once on a reference tenant; a tenant overrides only the locator plans or URL templates its configuration changes. Case 19 shows exactly where an override belongs.
Per-tenant reliability and approval	Designed Already tracked per version; would be tracked per (tenant, version), so one tenant's drift doesn't block the others.

10 Results and evidence

Genuine LLM discovery

`evidence/discovery-3EvQLgjD` : model `cohere/north-mini-code:free` drove a live browser for 6 tool-calling turns to look up a member and read two balances, then saved `lookup_member_balance` v1. In turn 5 the model replied in prose instead of calling a tool; the loop's guard caught that and the model called `finish` next.

Stability (multi-run replays, no LLM)

CAPABILITY	VERSION	MODE	RESULT	CONFIDENCE	TYPICAL TIME
`lookup_member_balance`	v1	live	10 / 10	72%	~730 ms
`lookup_member_balance`	v2	live	10 / 10	72%	~730 ms
`lookup_member_balance`	v3 (served)	live	10 / 10	72%	~730 ms
`open_member_subaccount`	v1	rehearsal	10 / 10	72%	~1.9 s
`open_member_subaccount`	v2 (served)	rehearsal	10 / 10	72%	~1.9 s

72% is the 95% lower bound on 10/10, which is the honest statement of what ten runs can prove. It clears the 70% approval policy.

Notable single runs

FOLDER	WHAT IT SHOWS
`replay-ijXh22BO`	Member 99999 → `MEMBER_NOT_FOUND` : an answer, not a crash
`replay-41DSaFxO`	Restricted member → `PERMISSION_DENIED`
`replay-1WmvA_Ov`	Core timeout once, retried by the artifact's own policy, then success
`replay-gCPvNgsx`	$5 deposit → `VALIDATION_ERROR` from the bank app
`replay-yGvbIgs4`	Paused at the irreversible step; the operator confirmed on the live page; success with a new account number
`replay-nSlenB5g`	A real human session: the operator pressed Resume without confirming. The run did not claim success, and this led to the decision model and page verification described in section 7.
`replay-x9ucrkEO`	Same v3 artifact against a second app instance with tenant B labels: fallback locator, drift recorded, success
`api-walkthrough-*`	The agent API over HTTP: catalog, invoke, idempotent retry and 409, bad input 400, 202 awaiting human, agent self-approval refused (403), no key (401), operator confirm and decline

Tests

64 automated tests in 12 files, all passing, no API key needed. They run the replay engine and the server against a real instance of the app. They cover:

- **Replay:** business outcomes, transient-failure recovery, escalation after retries run out, rehearsal.
- **Handoff:** confirm and decline on the same session, intervention timeout.
- **Server:** sign-in and roles, version serving, the approval gate, idempotency, capacity limits, input validation.
- **Unit tests:** the state machine, reliability scoring, contract checks and redaction.

11 Every case, with its result

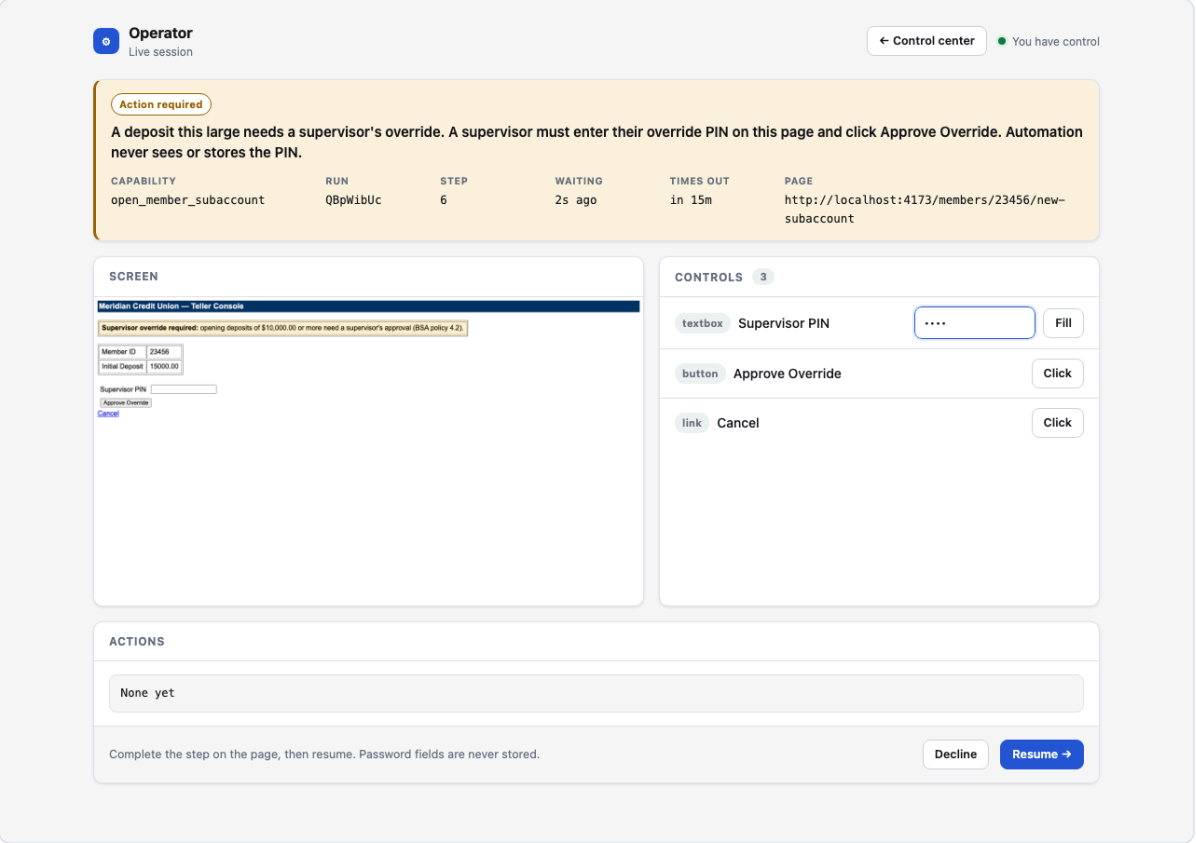
`npm run scenarios` runs one example of every case against the live app and checks each against the exact result it must produce. **29 / 29 pass.** These runs inject faults on purpose, so they are excluded from reliability scores.

#	KIND	CASE	EXPECTED AND ACTUAL RESULT
1	Success	Look up a member and return typed outputs	`success`
2	Success	Another tenant renamed a button; a fallback locator still finds it	`success` (drift recorded)
3	Success	Rehearse a risky flow without doing the irreversible thing	`rehearsal_complete`
4	Business	No such member	`MEMBER_NOT_FOUND`
5	Business	Teller not authorised for this member	`PERMISSION_DENIED`
6	Business	The bank app rejects the input ($5 deposit)	`VALIDATION_ERROR`
7	Recoverable	Core system times out once; retried automatically	`success`
8	Recoverable	A known compliance notice is dismissed	`success`
9	Recoverable	Retries run out; a person fixes it on the live page	`success`

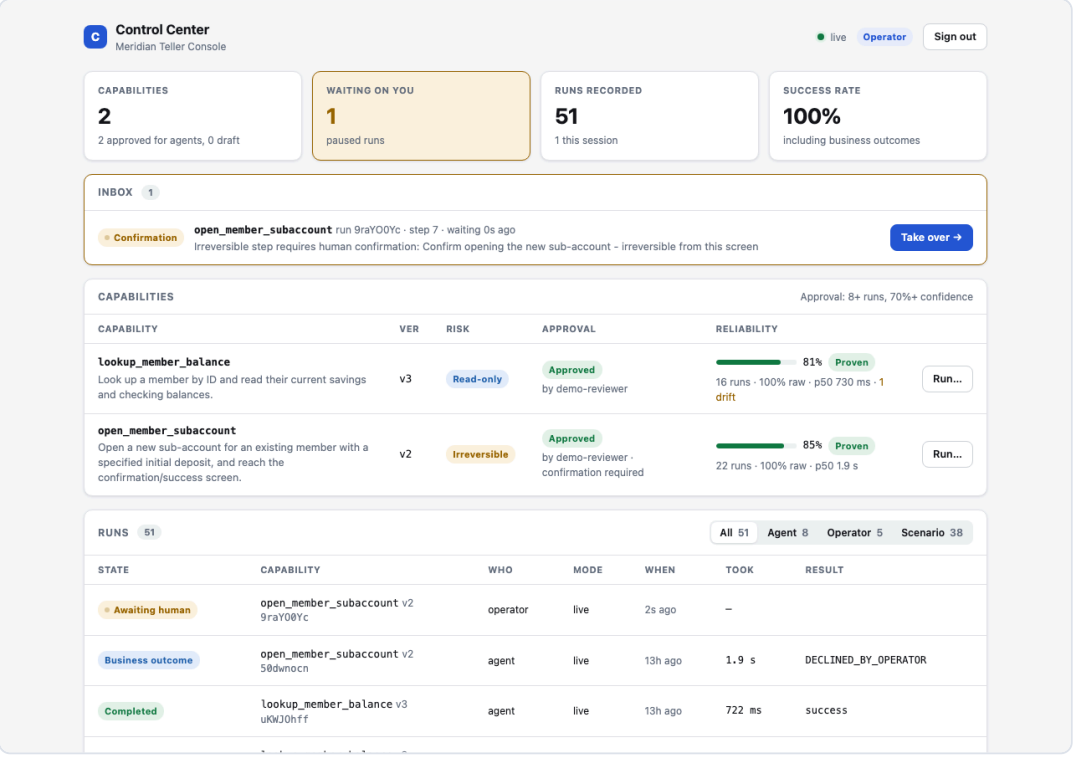
#	KIND	CASE	EXPECTED AND ACTUAL RESULT
10	Handoff	Irreversible step: a person confirms on the live session	`success`
11	Handoff	Irreversible step: the person declines	`DECLINED_BY_OPERATOR`
12	Handoff	Only a person may do it: supervisor override PIN for $15,000	`success` (PIN never recorded)
13	Handoff	The supervisor refuses the override	`DECLINED_BY_OPERATOR`
14	Handoff	Control handed back without confirming	`DECLINED_BY_OPERATOR`
15	Failure	Handed back but the required person's step wasn't done	`HUMAN_TASK_INCOMPLETE`
16	Failure	Nobody answers a pause	`OPERATOR_TIMEOUT`
17	Failure	Retries run out and the person gives up	`RECOVERY_EXHAUSTED`
18	Failure	Session expires mid-flow	`SESSION_EXPIRED`
19	Failure	A tenant changed a label no fallback covers	`LOCATOR_NOT_FOUND`
20	Failure	Malformed input (`memberId: "abc"`)	`INVALID_PARAMS`
21	Guardrail	A tampered artifact tries to leave the allowed app	`ORIGIN_NOT_ALLOWED`
22	Guardrail	A recording that ignores a caller's input is refused	rejected on save
23	Lifecycle	Approval refused without enough evidence	refused
24	Agent API	An agent discovers what it can call	approved tools only
25	Agent API	Invoke a capability by name with typed args	`200 COMPLETED`
26	Agent API	A retried request doesn't run twice	same run, then `409`
27	Agent API	An undeclared field (`ssn`) is rejected	`400 UNKNOWN_PARAMS`
28	Agent API	An agent can't approve its own irreversible action	`403, 403, 401`
29	Agent API	The agent learns the person's decision by long-polling	`DECLINED_BY_OPERATOR`

The stand-in app produces each condition on demand: member 40000 (restricted), 50000 (one timeout), 50002 (persistent outage), 60000 (session dies), 80000 (compliance notice), deposits of $10,000+ (supervisor PIN), and three tenant skins.

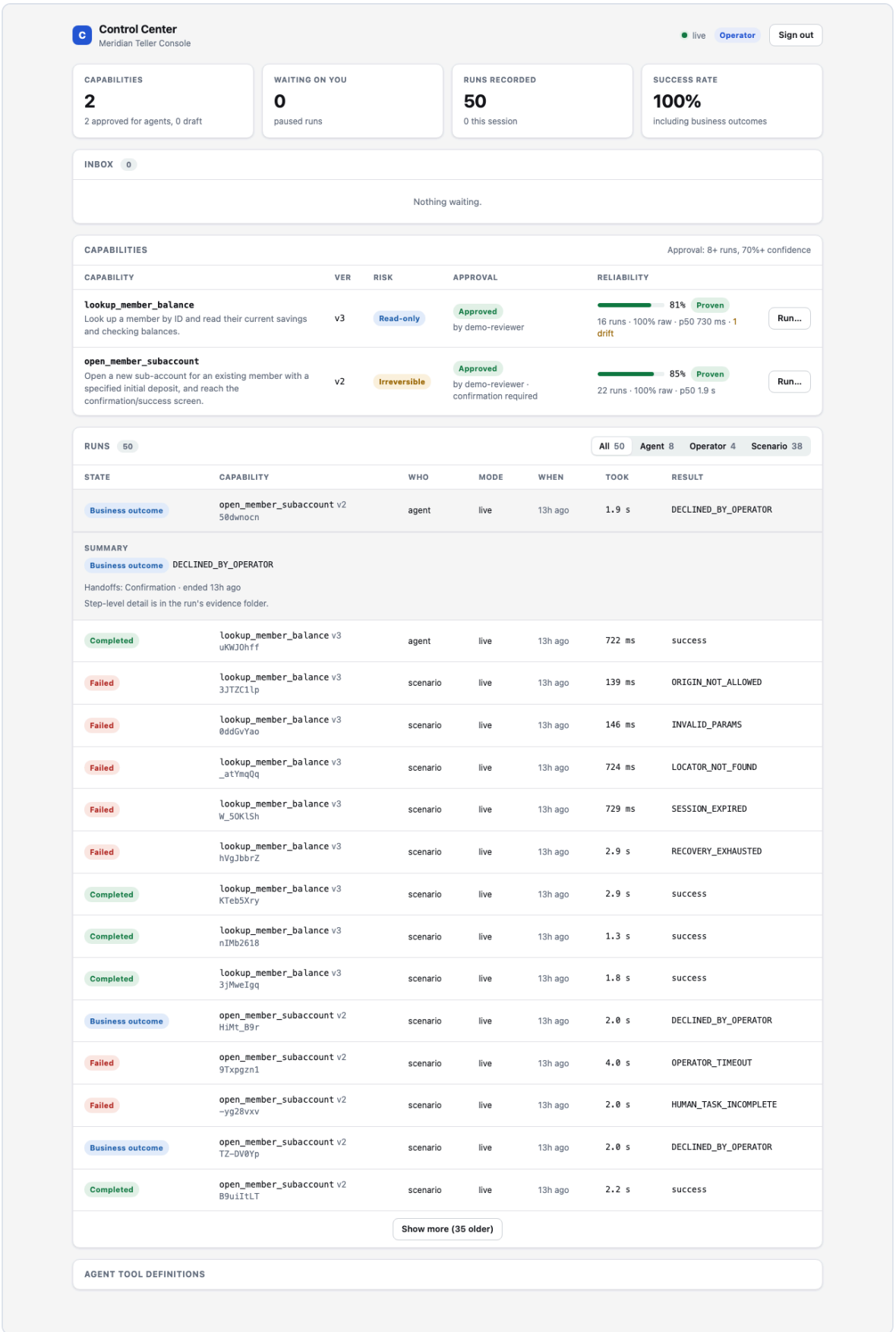
12 Screens



Operator takeover console. A $15,000 deposit needs a supervisor PIN. The left pane shows the live page; the right pane lists the controls the person can operate on that same page. The PIN field is masked and its value is never recorded. The person then resumes or declines. The deadline is shown under "Times out".



Control center inbox. A run waiting on a person, with the reason and a Take over button.



Control center. Capabilities with risk, approval state and reliability (confidence bar, run count, drift). Below, runs filtered by who started them, each expandable to its state timeline, step timings, handoffs and result.

13 What was cut, and honest gaps

Deliberately left out

CUT	WHY
Inferring outcomes and parameters automatically from one trace	Known outcomes are written per capability, and recorded values are matched to parameters. Doing this generically is a research problem; the contract check catches the dangerous failure (an ignored input).
Per-tenant overrides; desktop perception	The brief scopes these as design, not build. Both are designed (section 9); run-time origin and tenant variants are built.
Real-time co-browsing	Out of scope per the brief. The control-transfer mechanism is real; the view is a polled screenshot.
Durable runtime state, SSO, multi-host execution	Scaling infrastructure is explicitly not rewarded. The ledger and evidence are durable; live run state is in memory.
Code generation, assisted LLM fallback	The brief allows one or two stretch goals; two were chosen and done properly.

Honest gaps

- **Lost discovery evidence for `open_member_subaccount`.** Its artifact came from a genuine discovery run on 2026-09-27, but that run's folder was deleted by mistake on 2026-10-10 by an overly broad cleanup command, and no backup existed. The artifact and every replay of it survive. This is documented in `evidence/README.md`.
- **v2 of that capability was made by `revise`** from v1's recorded steps, not by a new model run. A fresh recording, and a run of `npm run agent-demo` (a real LLM choosing which tools to call), are both waiting on the free model tier's daily request limit.
- **Older evidence needed masking after the fact.** A sweep found member IDs in eight older files, written before the redaction fixes. They were masked in place and the cleanup is logged. Runs made with the current code had no leaks.
- **Approvals are signed by "demo-reviewer".** Re-run `npm run approve` with a real name to replace it.

Next, in order

1. Per-tenant overrides with per-tenant reliability: the brief's top priority for this environment.
2. Durable idempotency and run storage.
3. SSO-backed identities, so approvals and handoffs name the actual person.
4. Desktop perception through the OS accessibility tree.

14 How to run it

Setup

```
npm install
npx playwright install chromium
cp .env.example .env        # paste an OpenRouter key (only needed for discovery and agent-demo)
```

Demo path

```
# 1. the stand-in bank app (http://localhost:4173)
npm run target-app

# 2. discovery: an LLM records a capability (needs the key)
npm run discover -- --capability=lookup_member_balance --memberId=12345

# 3. deterministic replay, no LLM
npm run replay -- --capability=lookup_member_balance --memberId=23456     # success
npm run replay -- --capability=lookup_member_balance --memberId=99999     # MEMBER_NOT_FOUND
npm run replay -- --capability=open_member_subaccount --memberId=12345 --initialDeposit=75.00
#    pauses at the irreversible step: open http://localhost:4174 to confirm or decline

# 4. earn approval
npm run stability -- --capability=lookup_member_balance --runs=10 --memberId=12345
npm run approve   -- --capability=lookup_member_balance --reviewer="Your Name"

# 5. agents call capabilities by name; control center at http://localhost:4180
npm run server
npm run api-walkthrough

# every case, and the tests (no key needed)
npm run scenarios
npm test
```

Settings

VARIABLE	DEFAULT	PURPOSE
`OPENROUTER_API_KEY`	none	Discovery and agent-demo only; read from `.env`, which is not committed
`OPENROUTER_MODEL`	`cohere/north-mini-code:free`	Any tool-calling model
`TARGET_APP_URL`	`http://localhost:4173`	Where replays go; one artifact runs against any deployment
`INTERVENTION_TIMEOUT_MS`	900000 (15 min)	How long a pause waits for a person
`MAX_ACTIVE_RUNS`	4	Concurrent browser runs before `429`
`SERVER_PORT` / `OPERATOR_PORT`	4180 / 4174	Control center / CLI operator console

Sign in to the control center with the **operator** key from `.keys.json`, which is created on the first `npm run server`. Agents use the **agent** key as a Bearer token. Guardrail settings live in `config/allowlist.json`.